DESENVOLVIMENTO DE SOFTWARE VISUAL

PokéDraft

Guia da estrutura do projeto

O que é cada pasta, o que cada arquivo faz e os conceitos essenciais para começar a mexer no código.

Base: 1ª entrega (aula 28/08) — solução criada, banco modelado, primeira migration aplicada.

1. O que é o sistema

PokéDraft é um simulador de batalha Pokémon entre **dois jogadores**. O jogo tem três fases:

- **Draft de Pokémon** — o sistema sorteia vários *samples* (pacotes) de Pokémon, iguais para os dois jogadores. Cada jogador tem um orçamento em **pontos**; cada Pokémon tem um **custo**. Dá para **banir** e **comprar** Pokémon. Se os pontos acabam, o jogador pode **vender** um Pokémon do time (recebe menos do que pagou) para liberar pontos.
- **Draft de itens** — um único pacote com todos os itens de batalha, compartilhado. Cada item pode ser **equipado** em um Pokémon, somando ao **overall** dele.
- **Simulação da batalha** — cada jogador escolhe a ordem do time. Os times se enfrentam **1×1** (1º contra 1º, etc.). Cada duelo vencido vale 1 ponto; quem fizer mais pontos vence. Empate 3–3 → decide a média do overall.

Overall = status base do Pokémon + item equipado + natureza + IV. Na batalha entram bônus por vantagem de tipagem, velocidade e tipo de ataque.

Fora do escopo: várias gerações, draft de golpes (moveset), escolher IV/natureza manualmente.

2. Tecnologias e por quê

Camada	Tecnologia	Papel
Backend	C# / ASP.NET Core MVC (.NET 8)	Roda no servidor: regras do jogo, acesso ao banco, respostas.
API	Controllers REST (JSON)	Endpoints `GET/POST/PUT/DELETE` que o frontend chama.
Banco	SQLite + Entity Framework Core 8	Um arquivo `pokedraft.db`. O EF Core traduz classes C# ↔ tabelas.
Frontend	HTML + CSS + JavaScript (`fetch`)	Telas na pasta `wwwroot/` e `Views/`, consumindo a API.
Dados	PokéAPI	Fonte dos Pokémon (status, tipos, sprites) — importados para o nosso banco.
Apoio	Git + GitHub, Trello	Versionamento do código e quadro de tarefas.

> **SQLite** foi escolhido no lugar do MySQL: não precisa instalar nada nem subir servidor de banco — é só um arquivo. Cada integrante gera o seu localmente a partir das *migrations*.

3. Como rodar (primeira vez)

Pré-requisitos por máquina: **.NET SDK 8**, **Git**, **VS Code** (extensões **C#** e **C# Dev Kit**) e a ferramenta do EF Core:

```
dotnet tool install --global dotnet-ef --version 8.*
```

Depois:

```
git clone https://github.com/eduardowoz/PokeDraft-DesenvolvimentoSoftwereVIsual.git
cd PokeDraft-DesenvolvimentoSoftwereVIsual
dotnet restore
dotnet ef database update --project PokeDraft.Web   # cria o pokedraft.db
dotnet run --project PokeDraft.Web
```

O terminal mostra um endereço (`http://localhost:5xxx`) — abra no navegador.

4. Mapa de pastas

```
PokeDraft-DesenvolvimentoSoftwereVIsual/
├─ PokeDraft.sln            Solução: agrupa o(s) projeto(s)
├─ global.json              Trava o SDK em .NET 8 (todas as máquinas iguais)
├─ README.md                Setup, como rodar, fluxo de equipe
├─ .gitignore               O que o Git ignora (bin, obj, *.db...)
├─ .gitattributes           Normaliza fim de linha entre Windows/Mac/Linux
│
└─ PokeDraft.Web/             O projeto web (backend + telas + estáticos)
   ├─ PokeDraft.Web.csproj   Definição do projeto e pacotes NuGet
   ├─ Program.cs             Liga tudo: middlewares + registra o banco
   ├─ appsettings.json       Configuração (connection string do SQLite)
   ├─ appsettings.Development.json  Configuração só do ambiente de dev (logs)
   │
   ├─ Controllers/           Recebem requisições e respondem (MVC + API)
   │  └─ HomeController.cs
   │
   ├─ Models/
   │  ├─ Entities/           Classes do domínio — cada uma vira uma tabela
   │  │  ├─ Pokemon.cs      Item.cs         Jogador.cs
   │  │  ├─ Configuracao.cs Partida.cs      PokemonNoPool.cs
   │  │  ├─ ItemNoPool.cs   EscolhaPokemon.cs
   │  │  └─ MovimentacaoDraft.cs  ConfrontoBatalha.cs
   │  ├─ Enums/              Listas de valores fixos
   │  │  ├─ TipoPokemon.cs  StatusPartida.cs  TipoMovimentacao.cs
   │  └─ ErrorViewModel.cs  Model da tela de erro (do template)
   │
   ├─ Data/
   │  └─ PokeDraftDbContext.cs  A "ponte" entre as classes e o banco
   │
   ├─ Migrations/           Histórico de mudanças do banco (versionado!)
   │  ├─ 20260828125500_InicialCreate.cs
   │  ├─ 20260828125500_InicialCreate.Designer.cs
   │  └─ PokeDraftDbContextModelSnapshot.cs
   │
   ├─ Views/                HTML dinâmico (.cshtml)
   │  ├─ Home/Index.cshtml   Home/Privacy.cshtml
   │  ├─ Shared/_Layout.cshtml       Moldura comum a todas as páginas
   │  ├─ Shared/_Layout.cshtml.css   Shared/Error.cshtml
   │  ├─ Shared/_ValidationScriptsPartial.cshtml
   │  ├─ _ViewImports.cshtml usings globais das views
   │  └─ _ViewStart.cshtml   define o layout padrão
   │
   ├─ wwwroot/              Arquivos servidos direto ao navegador
   │  ├─ css/site.css        js/site.js        favicon.ico
   │  └─ lib/                Bootstrap + jQuery (bibliotecas prontas)
   │
   ├─ Properties/launchSettings.json  Portas/URLs ao rodar local
   └─ pokedraft.db           O banco (NÃO vai pro Git — cada um gera o seu)
```

5. Arquivo por arquivo

Raiz do repositório

Arquivo	O que faz
`PokeDraft.sln`	"Solução" do Visual Studio: uma lista de projetos que fazem parte do sistema. Hoje só tem o `PokeDraft.Web`. É o que você abre no VS.
`global.json`	Fixa a versão do .NET em **8**. Sem isso, um PC com .NET 10 poderia compilar diferente de um com .NET 8.
`README.md`	Instruções do projeto (pré-requisitos, como rodar, como trabalhar em equipe). É a primeira coisa que aparece no GitHub.
`.gitignore`	Lista de coisas que o Git **não** versiona: pastas `bin/` e `obj/` (resultado da compilação) e o arquivo `*.db`.
`.gitattributes`	Padroniza o "fim de linha" dos arquivos. Evita que o Git marque um arquivo inteiro como modificado só porque foi salvo em outro sistema operacional.

Configuração do projeto

Arquivo	O que faz
`PokeDraft.Web.csproj`	O "coração" do projeto: qual versão do .NET usar e quais **pacotes NuGet** estão instalados (aqui: `EntityFrameworkCore.Sqlite`, `.Design` e `.Tools`).
`Program.cs`	Ponto de partida da aplicação. Registra os serviços (Controllers, Views e o **banco de dados** via `AddDbContext<PokeDraftDbContext>` com SQLite) e monta o pipeline HTTP (rotas, arquivos estáticos, etc.).
`appsettings.json`	Configurações da aplicação em formato JSON. O mais importante para nós é a **connection string**: `"PokeDraftDb": "Data Source=pokedraft.db"` — onde fica o arquivo do banco.
`appsettings.Development.json`	Configurações que valem **só em desenvolvimento** (basicamente o nível de detalhe dos logs).
`Properties/launchSettings.json`	Define os endereços/portas usados quando você roda `dotnet run` localmente. Não afeta produção.

Backend — Controllers

Arquivo	O que faz
`Controllers/HomeController.cs`	Controller da página inicial (veio do template). Um **Controller** recebe a requisição do navegador, busca o que precisa e decide o que devolver (uma View ou um JSON). É aqui que vão nascer `PokemonController`, `PartidaController`, etc.

Backend — Models (o domínio)

Cada classe em `Models/Entities/` representa uma "coisa" do sistema e vira uma **tabela** no banco. As propriedades viram **colunas**.

Classe	Representa	Campos principais
`Pokemon`	Um Pokémon do catálogo (vem da PokéAPI).	Nome, Tipo1/Tipo2, os 6 status base, Geração, SpriteUrl, Custo, PokeApiId.
`Item`	Um item de batalha do catálogo.	Nome, Descrição, Categoria, BonusOverall, Custo.
`Jogador`	Uma pessoa que joga partidas.	Nome, CriadoEm.
`Configuracao`	Um "preset" de regras do draft, reutilizável.	BudgetPontos, QuantidadeSamples, PokemonPorSample, TamanhoTime, BansPorJogador, FatorVenda.
`Partida`	Uma disputa entre 2 jogadores. É a entidade central.	Status (enum), JogadorA/JogadorB, Configuração, Vencedor, PontosJogadorA/B.
`PokemonNoPool`	Um Pokémon sorteado para um sample do draft de uma partida.	PartidaId, PokemonId, NumeroSample, Banido, CompradoPorJogadorId.
`ItemNoPool`	Um item disponível no pacote de itens de uma partida.	PartidaId, ItemId, AdquiridoPorJogadorId.
`EscolhaPokemon`	Um Pokémon que um jogador comprou e pôs no time.	Pokémon, Item equipado, Natureza + 6 IVs, OrdemNaBatalha, PontosGastos, Vendida.
`MovimentacaoDraft`	Registro de cada ação no draft (histórico/auditoria).	Tipo (enum), Pokémon/Item, VariacaoPontos, SaldoResultante, Momento.
`ConfrontoBatalha`	Um duelo 1×1 dentro da batalha simulada.	Ordem, EscolhaPokemonA/B, OverallA/B, VencedorJogadorId.

E os `Models/Enums/` (listas de valores fixos):

Enum	Valores
`TipoPokemon`	Os 18 tipos (Normal, Fogo, Água, ...). Usado para calcular vantagem de tipagem.
`StatusPartida`	Configuracao → Draft → Batalha → Finalizada.
`TipoMovimentacao`	Banimento, CompraPokemon, VendaPokemon, EquiparItem, RemoverItem.
`ErrorViewModel`	(em `Models/`) model da tela de erro padrão — do template, não mexemos.

Backend — Data e Migrations

Arquivo	O que faz
`Data/PokeDraftDbContext.cs`	A **ponte** entre as classes e o banco. Tem um `DbSet<T>` por entidade (= uma tabela), configura os **relacionamentos** (quem aponta para quem, o que acontece ao apagar), os **índices** e um **dado inicial** (uma `Configuracao` "Padrão").
`Migrations/…_InicialCreate.cs`	A primeira **migration**. Método `Up()` = comandos que criam todas as tabelas; `Down()` = como desfazer. Gerado automaticamente pelo comando `dotnet ef migrations add`.
`Migrations/…_InicialCreate.Designer.cs`	Metadados dessa migration (usado pelo EF Core internamente). Não se edita à mão.
`Migrations/PokeDraftDbContextModelSnapshot.cs`	"Foto" do estado atual do modelo. O EF compara com as suas classes para descobrir o que mudou quando você cria a próxima migration.

Frontend — Views

Arquivo	O que faz
`Views/Home/Index.cshtml`	A página inicial. Hoje só mostra um texto de boas-vindas do PokéDraft.
`Views/Home/Privacy.cshtml`	Página de privacidade (do template).
`Views/Shared/_Layout.cshtml`	A **moldura** HTML comum a todas as páginas: `<head>`, menu de navegação, rodapé e o ponto onde o conteúdo de cada página entra (`@RenderBody()`).
`Views/Shared/_Layout.cshtml.css`	CSS que vale só para o layout.
`Views/Shared/Error.cshtml`	Tela mostrada quando dá erro não tratado.
`Views/Shared/_ValidationScriptsPartial.cshtml`	Bloco de scripts para validação de formulários no navegador.
`Views/_ViewImports.cshtml`	`@using` e "tag helpers" que valem para todas as views (evita repetir em cada arquivo).
`Views/_ViewStart.cshtml`	Diz qual é o layout padrão (`_Layout`) de todas as views.

Frontend — wwwroot

Arquivo	O que faz
`wwwroot/css/site.css`	CSS do site (nossas customizações de estilo vão aqui).
`wwwroot/js/site.js`	JavaScript do site. É daqui que vamos chamar a API com `fetch()`.
`wwwroot/favicon.ico`	Ícone da aba do navegador.
`wwwroot/lib/`	Bibliotecas prontas: **Bootstrap** (estilos) e **jQuery**. Vieram com o template; não editamos.

> **Regra de ouro do MVC:** só o que está em `wwwroot/` o navegador acessa diretamente. CSS/JS fora dessa pasta não carregam.

6. As tabelas do banco (resumo)

Tabela	Papel no jogo
`Pokemons`	Catálogo de Pokémon (importado da PokéAPI).
`Itens`	Catálogo de itens de batalha.
`Jogadores`	As pessoas que jogam.
`Configuracoes`	Presets de regras (já vem uma "Padrão" cadastrada).
`Partidas`	Cada disputa entre 2 jogadores.
`PokemonNoPool`	Pokémon sorteados nos samples de cada partida.
`ItensNoPool`	Itens disponíveis no pacote de itens de cada partida.
`EscolhasPokemon`	Os Pokémon que cada jogador comprou (com item, natureza, IVs, ordem).
`MovimentacoesDraft`	Histórico de ban / compra / venda / equipar item.
`ConfrontosBatalha`	Os duelos 1×1 da batalha simulada e seus resultados.
`__EFMigrationsHistory`	Controle interno do EF: quais migrations já foram aplicadas.

7. Conceitos rápidos

MVC (Model — View — Controller)

Jeito de separar responsabilidades: **Model** = os dados e as regras · **Controller** = recebe o pedido e coordena · **View** = o HTML que o usuário vê. Analogia do restaurante: cozinha (Model), garçom (Controller), mesa servida (View).

Entity Framework Core

É o **ORM**: em vez de escrever SQL na mão, você mexe em objetos C# e o EF traduz para o banco. `_context.Pokemons.Add(p)` + `SaveChanges()` vira um `INSERT`.

Migration

Um "commit" do formato do banco. Fluxo: você muda/cria classes em `Models/Entities/` → roda os comandos abaixo → o EF gera o código e aplica no seu `pokedraft.db`.

```
dotnet ef migrations add NomeDaMudanca --project PokeDraft.Web
dotnet ef database update --project PokeDraft.Web
```

> A pasta `Migrations/` **vai para o Git** junto com a mudança nas classes. O arquivo `pokedraft.db` **não vai**. Quem dá `git pull` e recebe migrations novas roda só `dotnet ef database update`.

API + fetch

O Controller devolve **JSON**; o JavaScript no navegador chama com `fetch()` e monta a tela. Verbos: `GET` busca · `POST` cria · `PUT` atualiza · `DELETE` remove. A rota no JS tem que ser **idêntica** à rota do Controller.

```
const resp = await fetch("/api/pokemons");
const lista = await resp.json();
```

Git — o básico do dia a dia

```
git pull                      # traz o que os colegas subiram
git checkout -b feat/minha-tarefa   # cria um branch para a tarefa
# ... edita, testa com: dotnet build ...
git add .
git commit -m "mensagem clara do que foi feito"
git push -u origin feat/minha-tarefa # sobe o branch
# abre um Pull Request no GitHub para juntar na main
```

8. Fluxo de trabalho da equipe

- A branch `main` tem que **sempre compilar e rodar**. Ninguém commita direto nela — cada tarefa em um branch + Pull Request.
- Antes de commitar: `dotnet build` sem erros.
- Deu `git pull` e veio migration nova? Rode `dotnet ef database update`.
- Cada card do Trello = uma tarefa de uma aula. Mova o card para "Em desenvolvimento" quando começar e para "Concluído" quando o PR entrar.

9. Próximas aulas

Data	Entrega
04/09	Importador da PokéAPI + CRUD de Pokémon na API
11/09	CRUD de Jogador, Partida e Configuração + esqueleto do frontend
18/09	**Marco:** protótipo navegável — criar partida e sortear o pool
02/10	Motor do draft: ban, compra, validações, controle de saldo
09/10	Venda + trava anti-beco-sem-saída + histórico de movimentações
16/10	**Marco:** MVP integrado — tela do draft ponta a ponta
23/10	Polimento visual, sprites, feedback de erro, responsividade
30/10	**Marco:** versão candidata — testes, casos de borda, bugs
06/11	Congelamento do código, README final, ensaio da defesa

Repositório: github.com/eduardowoz/PokeDraft-DesenvolvimentoSoftwereVIsual · Trello: trello.com/b/oK9mxrKm